



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Parallel Programming Assignment 7: Synchronization and Resource Sharing Spring Semester 2026

Assigned on: **01.04.2026**

Due by: (Wednesday Exercise) **06.04.2026**
(Friday Exercise) **08.04.2026**

Overview

In this exercise, we look at one of the major problems that arise in writing parallel programs: Accessing state that is shared between multiple threads.

Getting Prepared

- Fetch the new assignment from Gitlab by running:

```
git fetch upstream  
git merge upstream/main
```

You should now see the new assignment appear in your local directory.

- Open the project in IntelliJ IDEA: Click on *File* in the top-menu, then select *Open*. Select the folder of the new assignment.

Banking System

The CEO of the bank you are working for has recently bought some expensive multi-core machines. The vendor ensured the CEO that their four core machines will quadruple the number of transactions per second the bank can handle. After they installed the new machines, the CTO realized that their code base was not able to exploit the parallelism offered by their new servers. In order to keep their job, they called you for help.

The code skeleton you downloaded and imported contains the relevant parts of the KBS code base. There are two classes, `Account` and `BankingSystem`. `Account` represents the bank account of a client, while `BankingSystem` provides the essential functionality of the bank such as `transferMoney`, to transfer money between two accounts, and `totalMoneyInBank`, used by them to figure out how much money they currently maintain.

In the following tasks, we will first try to make `Account` and `BankingSystem` thread-safe, after that we will continue to increase the transaction performance of the system.

Task 1 - Problem identification: When the CTO installed the codebase on their new machines, they wanted to test the performance of the system. They wrote two simple tests (see `BankingThroughputTest`) that spawn a few threads and run a fixed number of transactions on random accounts. On one thread, everything worked fine (`testTransactionThroughputSingle`). However as soon as they started using multiple threads (`testTransactionThroughputParallel`), sometimes the test failed by triggering the following assertion on:

```
assertThat("Did not lose any money.", bs.totalMoneyInBank(), is(sum));
```

Try to reproduce the behavior on your machine. Understand what the test does by looking at the provided classes in `src` and `test`, then explain what went wrong and why (by providing an exact set of steps that can lead to such a scenario).

Task 2 - Synchronized: Having heard that the `synchronized` keyword is something to use in combination with threads, the CTO changed the signature of `transferMoney` in the `BankingSystem` class by adding `synchronized`:

```
public synchronized boolean transferMoney(Account from, Account to, int amount)
```

Now, the test did no longer fail, but, unfortunately, the performance reported by the parallel test-case was much worse than the sequential version:

```
Sequential using 1 Threads:
Completed 80000000 transaction in 1.920 sec: 4.1662e+07 transactions/sec
Parallel using 8 Threads:
Completed 80000000 Transaction in 13.443 sec: 5.9510e+06 transactions/sec
```

Because the CTO could not figure out what the problem is, you should explain them why the parallel version is slower than the sequential version, what the problem with the current code is and propose a way to fix it.

Task 3 - Locking: Impressed with your explanations in the previous task, the CTO thinks you should go ahead and implement your proposed changes to the `BankingSystem` and `Account` class such that it continues to be thread-safe but the transaction performance improves considerably with more threads on a multi-core machine. You are free to change and add code in any way, but be sure to keep the interface of `BankingSystem` the same for unit testing. Also, make sure that the implementation still corresponds to the one described in the Javadoc headers.

Try to answer the following questions:

Question 1: What if a transaction happens from and to the same account ?

```
transferMoney(a, a, X);
```

If this is a problem in your implementation, try to correct your code such that this is no longer an issue.

Question 2: Explain what measures you took in order to ensure that your code does not suffer from deadlocks.

Task 4 - Summing up: The method `sumAccounts` in the `BankingSystem` class is supposed to return the sum of money of all the accounts provided in the argument. However, at some point the CTO noticed that this function sometimes returns incorrect results, especially when a lot of transactions are happening concurrently. The CTO promises you a huge bonus if you can fix the function, such that it returns the correct amount of money at any point in time (i.e., while transactions are happening concurrently).

Question 3: Find out what is wrong with the current implementation by describing a scenario that can lead to an incorrect summation of the accounts.

Question 4: Change the implementation such that it now works for an arbitrary number of accounts.

Question 5: Are there ways to parallelize the summation? If so, describe how you could do it.

Submission

You need to submit your code and reports by using our submission system. This step will be the same for all of the subsequent exercises, but we will explain it in detail here so you can familiarize yourself with the system.

To submit your code using the terminal, follow the instructions below:

- In the terminal, navigate to your personal repository.
- Add your changes:
`git add .` # this command adds all changes in the git repo
- Commit your changes. In your commit message, make sure to include the [Submission] tag.
`git commit -m "[Submission] ..."`
- Push your changes!
`git push`

To submit your code using IntelliJ, follow the instructions below:

- Press the “Commit” button (found on the left hand toolbar)
- Select all the files you want changed during the assignment.
- Write a descriptive commit message. Make sure to include the [Submission] tag.
- Press “Commit and Push”

After this step, you can check your repository on the GitLab website in your browser. Make sure that you see your changes there!

Troubleshooting

Git Issues

In rare cases, you might get an error like this:

```
Can't connect to any URI
https://gitlab.inf.ethz.ch/COURSE-PPROG26/pprog26-<username>.git
(https://gitlab.inf.ethz.ch/COURSE-PPROG26/pprog26-<username>.git:
https://gitlab.inf.ethz.ch/COURSE-PPROG26/pprog26-<username>.git/info/refs?
service=git-receive-pack not found)
```

It means that no repository was created for your username yet. If that is the case, please contact your teaching assistant. The teaching assistant will be able to create the repository location for you. After that, you should be able to submit your work by following the instructions above.

IntelliJ / Java Issues

If you have trouble running the Java programs in IntelliJ, check the following:

- You have marked the `src` directory as a “Source Root”. Right click on the folder `src`, select “Mark Directory As >Source Root”.
- You have installed an updated JDK. We recommend `openjdk-25`. You can check this under “File >Project Structure >SDK”